Pourquoi Python affiche-t-il 0.30000000000000004 ?

Fiche de cours • Représentation des nombres à virgule en binaire • Exemple : 0,1 + 0,2

> **Objectif : comprendre pourquoi une addition mathématiquement exacte peut donner un résultat légèrement différent dans un ordinateur.**

1. Écrivons 0,1 et 0,2 en binaire

En base 10, nous écrivons 0,1 et 0,2 avec un nombre fini de chiffres. En base 2, leurs écritures sont infinies et périodiques :

```
0,1 = (0,0001100110011001100110011...)₂
0,2 = (0,0011001100110011001100110...)₂
```

Le groupe de quatre chiffres 0011 se répète indéfiniment. Ces nombres ne peuvent donc pas être stockés exactement avec un nombre fini de bits.

2. Additionnons les développements binaires infinis

```
  0,00011001100110011001100110011...
+ 0,00110011001100110011001100110...
  ---------------------------
  0,01001100110011001100110011001...
```

On retrouve le développement binaire de 0,3. Les retenues sont prises en compte comme dans une addition décimale.

3. Vérifions le résultat grâce à une série géométrique

Dans le résultat, le bloc 1001 (égal à 9 en base 10) revient tous les quatre rangs. On peut donc écrire :

$$0{,}3 = 9/2^5 + 9/2^9 + 9/2^{13} + \ldots$$
$$= (9/2^5) \times [1 + 1/2^4 + 1/2^8 + \ldots]$$

L’expression entre crochets est une série géométrique de raison 1/16. Sa somme vaut $1/(1 - 1/16) = 16/15$.

$$\text{Somme} = (9/32) \times (16/15) = 3/10 = 0{,}3$$

> **Premier constat : dans les nombres réels, 0,1 + 0,2 = 0,3 exactement. Il n’y a aucune erreur dans cette addition.**

Pourquoi le calcul de Python est-il différent ?

4. Un ordinateur conserve un nombre limité de bits

Le type float de Python utilise habituellement le format IEEE 754 « double précision », avec 53 bits significatifs. Il doit donc arrondir les écritures binaires infinies de 0,1 et de 0,2.

Les valeurs réellement mémorisées sont les fractions exactes suivantes :

```
0,1 stocké = 3602879701896397 / 2⁵⁵
0,2 stocké = 3602879701896397 / 2⁵⁴
```

Ces valeurs sont extrêmement proches des décimaux souhaités, mais ne leur sont pas exactement égales.

5. Additionnons les deux valeurs mémorisées

Après avoir aligné les chiffres binaires, on obtient :

```
    0,0001100110011001100110011001100110011001100110011001100110011001101
  + 0,0011001100110011001100110011001100110011001100110011001100110011010
    ------------------------------------------------------------------
    0,0100110011001100110011001100110011001100110011001100110011001100111
```

Cette somme exacte des valeurs stockées vaut :

```
S = 10808639105689191 / 2⁵⁵
  ≈ 0,30000000000000001665...
```

6. Le dernier arrondi est décisif

Autour de 0,3, deux nombres flottants consécutifs sont séparés de 2^{-54}. Or S tombe exactement à mi-distance entre les deux flottants suivants :

```
Flottant inférieur : 5404319552844595 / 2⁵⁴
Flottant supérieur : 5404319552844596 / 2⁵⁴
```

La règle IEEE 754 est : arrondir au plus proche ; en cas d’égalité, choisir celui dont le dernier bit significatif est pair. C’est ici le flottant supérieur qui est retenu.

7. Observons le résultat dans Python

```
>>> 0.1 + 0.2
0.30000000000000004
>>> (0.1 + 0.2) == 0.3
False
```

Python affiche la plus courte écriture décimale qui permet de retrouver le flottant calculé. Le nombre littéral 0.3 est, lui aussi, arrondi au stockage, mais vers le flottant inférieur.

> **À retenir : l’erreur ne vient pas de la règle d’addition, mais de la précision finie des nombres stockés et de l’arrondi du résultat.**